

Tekton-DAG: Stack-Aware Pull-Request Isolation for Polyglot Microservice Ecosystems

John Menke

Independent Researcher

Email: jmjava@gmail.com

ORCID: (to be added before camera-ready)

Abstract—Previewing a change to one service in a polyglot microservice stack usually means cloning an entire environment or hoping every hop forwards a routing key. We present *Tekton-DAG*, an open artifact that treats the application group as a directed acyclic graph and derives a single Tekton pipeline family from that graph. Pull requests rebuild only the changed node(s); W3C baggage and a session header identify the run; Telepresence or mirrord steal matching traffic while unmatched requests continue to hit the baseline deployment. Five language ecosystems ship originator/forwarder/terminal middleware that can be excluded from production builds. This 4-page tool paper states the problem, the model, the public artifact (platform repository, sample apps, Kind regression, narrated demos), and the functional validation we can currently defend—not an industrial case study. A 3–5 minute demonstration video and usage instructions accompany the submission. *Demo (Pages):* <https://jmjava.github.io/tekton-dag/> *Source:* <https://github.com/jmjava/tekton-dag> *Review video:* (YouTube URL, 3–5 min, to be inserted).

Index Terms—continuous integration, microservices, preview environments, Tekton, Kubernetes, W3C baggage, intercept routing

I. INTRODUCTION

Continuous integration is now ordinary industrial practice [1], [3], [4], yet a recurring gap remains when the *unit of change* is one repository inside a *runtime graph* of many services. Task DAGs in GitHub Actions, GitLab, Argo Workflows, or Tekton answer which *jobs* may run in parallel. They do not answer which *running service* must receive the pull-request (PR) binary, how a request still reaches that service through polyglot hops, or how baseline traffic avoids the PR pod.

Two common workarounds exist. *Clone-the-stack* preview environments (namespace- or review-app-per-PR) isolate well and scale poorly with stack width. *Mesh or flag routing* (Istio header match, feature-flag SDKs) isolates cheaply only if every constituent forwards the key—a social and technical problem across independently versioned codebases [6].

Tekton-DAG is a public, Kind-runnable platform that makes the *application DAG* the source of CI behavior. This paper is a **tool demonstration**: we describe envisioned users, the workflow, the artifact, and functional checks. We do *not* claim a controlled cost comparison against namespace cloning, nor an industrial deployment study; those are listed as future work.

a) Envisioned users.: Platform engineers who own a shared CI cluster; application teams who open PRs against

one service in a multi-repo stack; developers who attach a local process to an in-cluster graph (mirrord / Telepresence).

b) Contributions: (elaborated in Section III). (C1) Stack YAML as the sole driver of a four-pipeline family. (C2) Originator / forwarder / terminal session roles with five-framework libraries. (C3) Header-matched intercepts for `changed-app` only, with an original-traffic safety check, on two intercept backends. (C4) Optional test-to-node selection from a trace graph. (C5) A citable artifact: Apache-2.0 hub repo, sample apps, regression driver, and regenerable demos.

c) User workflow: (what the demonstration video walks through). (1) A platform engineer commits a stack file (apps, downstream edges, header name). (2) Bootstrap deploys the baseline graph once. (3) A developer opens a PR in *one* app repo; the orchestrator resolves the repo to a stack and sets `changed-app`. (4) The PR pipeline builds that node, attaches an intercept for the run header, validates that the header still appears along the chain, validates that unmatched traffic is unstolen, and runs the tests declared for the selected nodes. (5) Optionally the developer repeats the same header from an IDE via mirrord or Telepresence. (6) On merge, the release pipeline promotes the RC image and bumps the next development version.

II. RELATED WORK

a) CI and delivery.: Hilton et al. quantify OSS CI adoption and costs [1], [2]. Shahin et al. survey continuous practices [3]. Humble and Farley [4] and Bass et al. [5] set the process vocabulary. We assume CI is worthwhile; the gap is graph-aware PR isolation.

b) Task graphs, not application graphs.: Tekton Pipelines [7], Argo Workflows [8], and GitHub Actions `needs`: model job dependencies. Dagger and Earthly similarly encode build DAGs. Our DAG is the user architecture (`downstream`: edges); job order is derived.

c) Preview environments and meshes.: PaaS review apps and Kubernetes namespace-per-PR duplicate the graph. Istio and Envoy can match HTTP headers [9] if every sidecar and application cooperates. We keep one baseline stack and steal only matching requests (Section III-C).

d) Developer intercepts.: Telepresence [10] and mirrord [11] already implement header filters from a laptop. We place those dataplanes from CI, using the same header the DAG roles propagate, and add unmatched-traffic validation.

TABLE I
PR ISOLATION STRATEGIES (QUALITATIVE).

	Clone stack / namespace	Mesh header routing	This artifact
What is duplicated	Whole graph	Canary pods	PR pods for changed-app only
Who must cooperate	Cluster quota	Every sidecar + app	Apps with a propagation role
Baseline traffic	Separate copy	Requires careful matchers	Explicit unmatched probe

TABLE II
EXEMPLAR STACK-ONE (CALLER $\rightarrow$ CALLEE).

App	Role	Toolchain
demo-fe	originator	Vue / npm
release-lifecycle-demo	forwarder	Spring Boot / Maven
demo-api	terminal	Spring Boot / Maven

e) *Context propagation.*: W3C Trace Context and Baggage [12] and OpenTelemetry [13] standardize key propagation. Feature-flag SDKs thread targeting keys. We reuse baggage as a *dev-session* identifier with explicit graph roles and production-exclusion of the libraries.

f) *Test selection.*: Ekstazi [14] and related test-impact analysis operate on code dependencies. Our optional test-plan query (C4) maps *HTTP tests* to *service nodes* from traces. It is complementary, not a replacement for class-level RTS.

III. APPROACH

A. C1: Stack DAG as pipeline source

A *stack* is one YAML document: nodes (apps with repo, toolchain, tests) and caller $\rightarrow$ callee edges. The graph must be acyclic; resolve-time topological sort fails otherwise. From the graph the resolver emits topological order, the *entry* app (no inbound edge), the propagation chain, *build-apps* (the PR’s *changed-app* or the full topo list on merge), and a per-run intercept header (e.g. `x-dev-session: pr-42`).

Four pipelines share that JSON: *stack-bootstrap* (deploy the baseline once), *stack-pr-test* (build intercepts, validate, test), *stack-merge-release* (promote RC to release), and *stack-promote* (copy images to a target registry). Adding a stack does not add a pipeline YAML file.

Table II sketches the three-node exemplar *stack-one*. Table I places the strategy against the two workarounds from the introduction: we claim a *different isolation mechanism*, not a measured cost win.

B. C2: Three propagation roles

Traffic is tagged with a stack-configured header and optional W3C baggage key. **Originator** (usually the entry UI) sets the header on outbound calls. **Forwarder** accepts, stores in request context, and forwards. **Terminal** accepts and never forwards. Roles may be declared or inferred (no downstream $\Rightarrow$ terminal; frontend $\Rightarrow$ originator).

Standalone libraries live under `libs/` for Spring Boot, Java servlet/WAR, Node, Flask/WSGI, and PHP PSR-15. They are consumed as *dev* dependencies (Maven profile, `require-dev`, `devDependencies`) so production builds can omit them; runtime flags keep the code inert if accidentally packaged.

C. C3: Intercepts and original-traffic safety

On a PR the pipeline builds only *build-apps*, deploys PR pods, and attaches intercepts whose match is the run header. Unmatched requests must remain on the baseline Service. We implemented two backends behind *intercept-backend*: *Telepresence* (`-http-match`) and *mirrord* (`header_filter`). A dedicated task sends traffic *without* the header while intercepts are active (`validate-original-traffic`).

Local debugging uses the same header: *mirrord* or *Telepresence* can steal from a laptop against the in-cluster graph (VS Code launch configs in the hub repo).

D. C4: Test-to-node selection (optional)

Nightly traces (mock Datadog API) populate a Neo4j graph of tests versus nodes. The orchestrator exposes a test plan for *changed-app*. When no tests map, the area is flagged rather than silently skipped. We treat C4 as a capability in this paper, not as a measured improvement over full e2e suites.

E. Control plane

An in-cluster Flask orchestrator maps GitHub webhooks and REST calls to *PipelineRuns*; a Helm chart plus optional Argo CD ApplicationSet instantiates per-team namespaces. An optional Kubebuilder operator (*Stack / StackRun* CRDs) can own *PipelineRun* creation. These are packaging, not additional research claims.

IV. ARTIFACT

The hub repository `jmjava/tekton-dag` (Apache License 2.0) holds pipelines, stack schema, orchestrator, Helm chart, baggage libraries, regression scripts, and demo sources. Six sample application repositories under the same GitHub user form the runtime graph; they are cloned by the pipeline rather than vendored as submodules. Narrated segments (Manim + optional VHS + TTS) are published on GitHub Pages and can be regenerated with the `docgen` toolchain.

a) *Reviewer-timed path (no cluster).*:
`./scripts/run-regression.sh --local-only`
 checks stack YAML, the resolver, and unit tests. This is the path we expect demonstration reviewers to run, if any.

b) *Cluster path (not required of demo reviewers).*:
 Kind + local registry + Tekton install is documented in the README and `DO-THIS-LOCAL.md`. Full intercept E2E is `run-e2e-with-intercepts.sh`. That path exceeds a typical 30-minute artifact-evaluation budget unless a preloaded cluster image is provided later.

Citation metadata is in `CITATION.cff`. A Zenodo DOI of a tagged release is the remaining step for an ACM “Artifacts Available” badge; a GitHub URL is not archival [15].

V. PRELIMINARY VALIDATION

Claims are scoped to *functional* evidence.

a) *Header isolation.*: On Kind, against stack-one’s BFF, five unmatched HTTP requests stayed on the baseline pod and five header-matched requests were stolen by the intercept process [16]. The same header predicate is used for Telepresence. Scripts exercise the PR pipeline with `intercept-backend=telepresence` and `=mirrord`.

b) *Resolver and pipelines.*: Phase-1 scripts compare `stack-graph.sh` to stack YAML without a cluster; phase-2 waits for a `stack-dag-verify` PipelineRun to succeed and match CLI resolution. Several stack files (three-tier, vendor, single-app, mixed-hop) exist in-tree.

c) *Libraries and platform tests.*: Baggage packages have unit tests; mixed-role test stacks drive propagation validation. The hub regression driver layers `pytest`, `vitest`, `Playwright` (management GUI), `Newman` (orchestrator API), and optional Tekton Results checks. Counts drift with the tree; the submission tag should reprint numbers from that commit.

d) *What we do not report.*: No wall-clock or node-minute comparison versus namespace-per-PR; no concurrent multi-PR statistical study; no developer-user study; no TIA precision/recall. Stating those as results would be unsupported.

Local Kind runs and selective demo regeneration keep the evaluation footprint small; we do not report a kilogram-scale carbon figure.

VI. THREATS TO VALIDITY

Construct: isolation is HTTP routing, not storage or IAM isolation. *Internal*: PoC repetitions are small and author-run; Kind is not a production control plane; intercept agents require elevated capabilities. *External*: sample apps were written to fit the platform. *Reliability*: cluster E2E is timing-sensitive; we cite scripts, not a single laptop run.

VII. CONCLUSION

Tekton-DAG makes an application DAG the source of CI behavior so a PR can rebuild and intercept one node of a polyglot stack without cloning the world and without dropping the session header. The public artifact is intended as a workshop-quality exemplar: inspectable, Kind-reproducible at the unit/resolver layer, and demonstrated in short videos. Next steps that would lift the work toward Software Engineering in Practice or a research track are (i) a measured comparison of intercept versus namespace clone, (ii) concurrent-PR isolation tests, and (iii) a named industrial deployment.

ACKNOWLEDGMENT

Demo narration and some documentation were edited with generative writing tools; the author reviewed all technical claims against the repository. Affiliation and ORCID will be completed before camera-ready. This draft is not yet submitted.

REFERENCES

- [1] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, “Usage, costs, and benefits of continuous integration in open-source projects,” in *ASE*, 2016, pp. 426–437.
- [2] M. Hilton, N. Nelson, T. Tunnell, D. Marinov, and D. Dig, “Trade-offs in continuous integration: assurance, security, and flexibility,” in *ESEC/FSE*, 2017, pp. 197–207.
- [3] M. Shahin, M. A. Babar, and L. Zhu, “Continuous integration, delivery and deployment: a systematic review on approaches, tools, challenges and practices,” *IEEE Access*, vol. 5, pp. 3909–3943, 2017.
- [4] J. Humble and D. Farley, *Continuous Delivery*. Addison-Wesley, 2010.
- [5] L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect’s Perspective*. Addison-Wesley, 2015.
- [6] N. Dragoni *et al.*, “Microservices: yesterday, today, and tomorrow,” in *Present and Ulterior Software Engineering*. Springer, 2017, pp. 195–216.
- [7] Tekton Authors, “Tekton Pipelines,” <https://tekton.dev/>, 2026.
- [8] Argo Project, “Argo Workflows,” <https://argoproj.github.io/workflows/>, 2026.
- [9] Istio Authors, “Istio traffic management,” <https://istio.io/latest/docs/concepts/traffic-management/>, 2026.
- [10] Ambassador Labs, “Telepresence,” <https://www.telepresence.io/>, 2026.
- [11] MetalBear, “mirrord,” <https://mirrord.dev/>, 2026.
- [12] W3C, “Baggage,” W3C Proposed Recommendation, <https://www.w3.org/TR/baggage/>, 2022.
- [13] OpenTelemetry Authors, “OpenTelemetry specification,” <https://opentelemetry.io/docs/specs/otel/>, 2026.
- [14] M. Gligoric, L. Eloussi, and D. Marinov, “Practical regression test selection with dynamic file dependencies,” in *ISSTA*, 2015, pp. 211–222.
- [15] ACM, “Artifact review and badging version 1.1,” <https://www.acm.org/publications/policies/artifact-review-and-badging-current>, 2020.
- [16] J. Menke, “Milestone 5: MetalBear mirrord PoC results,” `tekton-dag repository, docs/mirrord-poc-results.md`, 2026.